ENTERPRISE KUBERNETES MASTERY

AI Platform Engineering Handbook

PART X OBSERVABILITY

OpenTelemetry, Prometheus, Grafana, Loki, Tempo, Cost and AI Observability

Volume 10 of 16 Advanced Series

Prerequisites: Parts I through IX

Edition 2025-2026

TABLE OF CONTENTS

CHAPTER 1

Observability Philosophy: The Three Pillars

Observability is the ability to understand the internal state of a system from its external outputs. In the context of Kubernetes and distributed systems, observability is not a nice-to-have -- it is the operational foundation that makes everything else possible: incident response, capacity planning, SLO compliance, security forensics, and cost optimisation.

The Three Pillars

* **Metrics**: Numeric time-series data describing system behaviour over time. Metrics are aggregated, low-cardinality, and cheap to store. They answer: Is the system healthy? Is it meeting its SLO? What is the trend? Implemented in Kubernetes via Prometheus.
* **Logs**: Discrete, timestamped records of events. Logs are high-cardinality, rich with context, and expensive to store. They answer: What exactly happened? What were the surrounding conditions? What is the error message? Implemented via Loki, Elastic, or cloud logging services.
* **Traces**: End-to-end records of requests flowing through distributed systems. Traces show causality: which service called which, what was the latency at each hop, where did the error originate? Implemented via OpenTelemetry, Tempo, or Jaeger.

The Four Golden Signals (Google SRE)

Signal	Metric Type	Kubernetes Metric Example	Alert Threshold
Latency	Histogram	http_request_duration_seconds_p99	p99 > 500ms for 5min
Traffic	Counter/Gauge	http_requests_total (rate)	Sudden drop > 50% from baseline
Errors	Counter	http_requests_total{status=~'5..'}	Error rate > 1% of traffic
Saturation	Gauge	container_cpu_usage / requests	CPU throttle > 25% of periods

Observability Stack Architecture

```
Enterprise Kubernetes observability stack: METRICS PIPELINE: Application -> Prometheus scrape -> Thanos -> Grafana -> Object store (S3) [long-term] LOG PIPELINE: Container stdout/stderr -> Fluent Bit (DaemonSet) -> Loki -> Grafana Or: Fluent Bit -> Elasticsearch -> Kibana (ELK) TRACE PIPELINE: Application (OTLP SDK) -> OpenTelemetry Collector -> Tempo -> Grafana Or: -> Jaeger EVENTS PIPELINE: Kubernetes Events -> kube-state-metrics -> Prometheus Kubernetes Audit -> Fluent Bit -> Loki AI/GPU METRICS: DCGM Exporter (DaemonSet) -> Prometheus -> Grafana GPU dashboards COST PIPELINE: OpenCost (DaemonSet) -> Prometheus -> Grafana cost dashboards
```

CHAPTER 2

OpenTelemetry: The Unified Instrumentation Standard

OpenTelemetry (OTel) is the CNCF standard for generating, collecting, and exporting telemetry data (metrics, logs, traces). It merges OpenCensus and OpenTracing into a single, vendor-neutral SDK and wire protocol (OTLP). For Kubernetes platforms, OpenTelemetry enables auto-instrumentation without code changes and provides a unified collection pipeline that routes to any backend.

OpenTelemetry Components

* **OTel SDK**: Language-specific libraries (Go, Java, Python, Node.js, Rust, .NET) that add instrumentation to application code. Provides APIs for spans, metrics, and logs. Auto-instrumentation libraries instrument popular frameworks (HTTP servers, database clients, message queues) without code changes.
* **OTel Collector**: A vendor-agnostic agent/gateway that receives telemetry from applications (via OTLP, Jaeger, Zipkin, Prometheus), processes it (batch, filter, transform, sample), and exports to backends. Deployed as a DaemonSet (agent mode) or centralised Deployment (gateway mode).
* **OTLP (OTel Protocol)**: The wire protocol for all OTel data. gRPC and HTTP/protobuf. Backends that support OTLP include: Tempo, Jaeger, Datadog, Dynatrace, New Relic, Honeycomb, and dozens more.

OTel Collector Deployment on Kubernetes

```
# OTel Collector as DaemonSet (agent mode -- per node): # Collects: host metrics, container logs, node-level telemetry apiVersion: opentelemetry.io/v1alpha1 kind: OpenTelemetryCollector metadata: name: otel-agent namespace: observability spec: mode: daemonset config: | receivers: otlp: protocols: grpc: { endpoint: 0.0.0.0:4317 } http: { endpoint: 0.0.0.0:4318 } hostmetrics: collection_interval: 30s scrapers: cpu: {} memory: {} disk: {} network: {} kubeletstats: auth_type: serviceAccount endpoint: https://localhost:10250 insecure_skip_verify: true processors: batch: timeout: 1s send_batch_size: 1024 memory_limiter: limit_mib: 512 k8sattributes: extract: metadata: [k8s.pod.name, k8s.namespace.name, k8s.deployment.name] exporters: otlp: endpoint: otel-gateway:4317 prometheusremotewrite: endpoint: http://thanos-receive:10908/api/v1/receive service: pipelines: traces: { receivers: [otlp], processors: [batch, k8sattributes], exporters: [otlp] } metrics: { receivers: [hostmetrics, kubeletstats], processors: [batch], exporters: [prometheusremotewrite] }
```

Auto-Instrumentation with OTel Operator

```
# Instrument a Python application without code changes: apiVersion: opentelemetry.io/v1alpha1 kind: Instrumentation metadata: name: python-instrumentation namespace: production spec: exporter: endpoint: http://otel-agent:4317 python: env: - name: OTEL_EXPORTER_OTLP_ENDPOINT value: http://otel-agent:4317 - name: OTEL_METRICS_EXPORTER value: otlp --- # Annotate Pod for auto-instrumentation injection: metadata: annotations: instrumentation.opentelemetry.io/inject-python: 'true' # OTel Operator injects init container that installs # opentelemetry-distro and configures OTLP exporter
```

CHAPTER 3

Prometheus: Metrics Collection at Scale

Prometheus is the de facto metrics system for Kubernetes. It uses a pull model (scraping HTTP endpoints) with a powerful time-series database and PromQL query language. Understanding Prometheus internals is essential for designing reliable, scalable monitoring at enterprise scale.

Prometheus Architecture

```
Prometheus components: Prometheus Server: Scrape engine: Discovers and scrapes /metrics endpoints TSDB: Time-series database (local, 2-byte aligned FLOAT64) Rules engine: Evaluates recording and alerting rules HTTP API: PromQL query API used by Grafana Service Discovery: kubernetes_sd: Discovers Pods, Services, Nodes via Kubernetes API file_sd: Discovers targets from JSON/YAML files ServiceMonitor: Prometheus Operator CRD for self-service scrape config Alertmanager: Routes alerts to: Slack, PagerDuty, OpsGenie, email, webhook Deduplication, silencing, inhibition, grouping Pushgateway: Accepts pushed metrics (for Jobs that cannot be scraped) Caution: not a replacement for scraping; use for batch Jobs only
```

Prometheus Operator and ServiceMonitors

```
# Install kube-prometheus-stack (Prometheus Operator + Grafana + node_exporter): helm repo add prometheus-community https://prometheus-community.github.io/helm-charts helm install kube-prometheus-stack \ prometheus-community/kube-prometheus-stack \ --namespace monitoring --create-namespace \ --set prometheus.prometheusSpec.retention=30d \ --set prometheus.prometheusSpec.retentionSize=100GB \ --set prometheus.prometheusSpec.storageSpec.volumeClaimTemplate.spec.storageClassName=fast-ssd \ --set prometheus.prometheusSpec.storageSpec.volumeClaimTemplate.spec.resources.requests.storage=200Gi # ServiceMonitor: self-service scrape registration: apiVersion: monitoring.coreos.com/v1 kind: ServiceMonitor metadata: name: myapp namespace: production labels: release: kube-prometheus-stack # Must match Prometheus selector spec: selector: matchLabels: { app: myapp } endpoints: - port: http path: /metrics interval: 15s scrapeTimeout: 10s metricRelabelings: - sourceLabels: [__name__] regex: 'go_.*' action: drop # Drop Go runtime metrics to save cardinality
```

Essential Kubernetes PromQL Queries

What to Monitor	PromQL Query
Pod CPU throttling rate	sum(rate(container_cpu_cfs_throttled_periods_total[5m])) by (pod,namespace) / sum(rate(container_cpu_cfs_periods_total[5m])) by (pod,namespace)
Memory usage vs limit	container_memory_working_set_bytes / container_spec_memory_limit_bytes
Pod restart rate	increase(kube_pod_container_status_restarts_total[1h]) > 3
Node disk pressure	kubelet_volume_stats_used_bytes / kubelet_volume_stats_capacity_bytes > 0.80
API server latency p99	histogram_quantile(0.99, sum(rate(apiserver_request_duration_seconds_bucket[5m])) by (le, verb))

What to Monitor	PromQL Query
HTTP error rate	sum(rate(http_requests_total{status=~'5..'}[5m])) / sum(rate(http_requests_total[5m]))
Deployment replica lag	kube_deployment_spec_replicas - kube_deployment_status_replicas_available

CHAPTER 4

Thanos: Long-Term Metrics Storage

Prometheus is excellent for short-term metrics (30-90 days local retention) but its single-node TSDB does not scale for enterprise requirements: multi-cluster queries, years of retention, high availability, and deduplication across replicas. Thanos solves all of these by adding a distributed layer over Prometheus.

Thanos Architecture

```
Thanos components: Sidecar (runs alongside each Prometheus): Uploads completed TSDB blocks to object storage (S3/GCS) Exposes Prometheus data via Store API for Querier Querier (Deployment): Receives PromQL queries Fans out to: all Sidecars, Store Gateways, Rulers Deduplicates results from Prometheus replicas Returns unified result Store Gateway (Deployment): Reads historical blocks from object storage Exposes via Store API to Querier Cache: in-memory + index cache (Memcached/Redis) Compactor (CronJob): Compacts small blocks into larger ones (1h->2h->8h->48h->...) Applies retention policy (delete blocks older than N days) Downsamples: 5m resolution (after 40d), 1h resolution (after 1y) Ruler (optional): Evaluates recording/alerting rules across long-term data Stores results back to object storage Receive (optional, push model): Accepts Prometheus remote_write Enables multi-cluster fan-in without Sidecar
```

Thanos Deployment Pattern

```
# Thanos Sidecar in Prometheus StatefulSet: containers: - name: prometheus image: prom/prometheus:v2.51.0 args: - --storage.tsdb.retention.time=2d - --storage.tsdb.min-block-duration=2h # Required for Thanos - --storage.tsdb.max-block-duration=2h # Required for Thanos - --web.enable-lifecycle - name: thanos-sidecar image: quay.io/thanos/thanos:v0.35.0 args: - sidecar - --tsdb.path=/prometheus - --prometheus.url=http://localhost:9090 - --objstore.config-file=/etc/thanos/objstore.yaml - --grpc-address=0.0.0.0:10901 volumeMounts: - name: prometheus-storage mountPath: /prometheus - name: objstore-config mountPath: /etc/thanos # objstore.yaml (S3 backend): type: S3 config: bucket: company-thanos-metrics region: us-east-1 endpoint: s3.amazonaws.com
```

CHAPTER 5

Grafana: Dashboards and Alerting

Grafana is the primary visualisation and alerting layer for the Prometheus/Loki/Tempo observability stack. In enterprise environments, Grafana serves as the unified observability portal: operators, developers, and executives all use Grafana dashboards to understand system health.

Essential Kubernetes Dashboards

Dashboard	Source	What It Shows
Kubernetes / Compute Resources / Node	kube-prometheus	CPU, memory, network per node; identify hot nodes
Kubernetes / Compute Resources / Namespace	kube-prometheus	Per-namespace resource usage vs quotas
Kubernetes / Compute Resources / Pod	kube-prometheus	Per-pod CPU throttling, memory, network
Kubernetes / Networking / Namespace	kube-prometheus	Bytes in/out per namespace, pod, service
Node Exporter Full	node_exporter	Detailed OS metrics: disk I/O, filesystem, network
NVIDIA DCGM Exporter Dashboard	NVIDIA	GPU utilisation, memory, temperature, power
OpenCost	OpenCost	Cost per namespace, workload, pod, GPU hour
ArgoCD	ArgoCD plugin	Application sync status, health, history
Loki Dashboard	Loki	Log volumes, error rates, log search

Grafana Alerting Configuration

```
# Grafana unified alerting rule (PromQL-based): apiVersion: 1 groups: - orgId: 1 name:
kubernetes-critical folder: Production Alerts interval: 1m rules: - uid: pod-oom-killed title:
Pod OOM Killed condition: C data: - refId: A model: expr: | sum by (namespace, pod, container)
( kube_pod_container_status_last_terminated_reason{reason='OOMKilled'} ) > 0 noDataState:
NoData execErrState: Error for: 1m annotations: summary: Pod OOM Killed description: Pod {{
$labels.pod }} in {{ $labels.namespace }} was OOM killed runbook_url:
https://wiki.company.com/runbooks/pod-oom-kill labels: severity: critical team: platform #
Contact points (notification channels): contactPoints: - name: pagerduty-critical receivers: -
type: pagerduty settings: integrationKey: XXXXXXXXXXXXXXXX severity: critical
```

CHAPTER 6

Loki: Log Aggregation for Kubernetes

Loki (Grafana Labs) is a horizontally scalable log aggregation system designed for Kubernetes. Unlike Elasticsearch (which indexes all log content), Loki only indexes labels and stores log chunks compressed in object storage. This makes Loki dramatically cheaper than Elasticsearch for high-volume Kubernetes log ingestion.

Loki Architecture

```
Loki components (microservices mode): Distributor: Receives log streams from Fluent Bit/Promtail; validates; shards Ingester: Buffers recent logs in memory; flushes to object storage Querier: Executes LogQL queries; reads from ingesters + object store Query Frontend: Request splitting, caching, parallelisation Ruler: Evaluates alerting rules on log streams Compactor: Retention policy; compacts chunks Storage: Index: DynamoDB, Cassandra, BoltDB, or filesystem (for small deployments) Chunks: S3, GCS, Azure Blob, or filesystem Scale mode options: monolithic: Single binary (dev, small clusters) simple-scalable: Read/write/backend split (medium clusters) microservices: Full component split (large enterprise)
```

LogQL -- Loki Query Language

```
# Log stream selector (label-based): {namespace='production', app='api-server'} # Filter for errors: {namespace="production"} |= "ERROR" | json # Parse structured JSON logs: {namespace="production", app="llm-inference"} | json | level="error" # Error rate over time (metric query): sum(rate({namespace="production"} |= "ERROR" [5m])) by (app) # Find slow requests (latency > 1s in structured logs): {app="api-server"} | json | duration > 1s # Pattern detection (find common patterns in unstructured logs): {namespace="production"} | pattern `<_> level= msg=` | level='error' | line_format '{{.message}}' # Alert rule on log error rate: groups: - name: loki-alerts rules: - alert: HighErrorRate expr: | sum(rate({namespace="production"} |= "ERROR" [5m])) by (app) > 10 for: 5m annotations: summary: High error rate in {{ $labels.app }}
```

CHAPTER 7

Tempo: Distributed Tracing

Distributed tracing follows a request as it flows through multiple services, recording the time spent at each hop and any errors encountered. In Kubernetes microservice environments, tracing is the only way to understand end-to-end latency and pinpoint bottlenecks across service boundaries.

Tracing Concepts

```
Trace: A directed acyclic graph of spans representing one request Span: A unit of work with: trace_id: Same across all spans in a trace span_id: Unique to this span parent_span_id: Link to calling span operation_name: What this span does (HTTP GET /api/v1/query) start_time + duration status: OK / ERROR attributes: key-value metadata (http.status_code, db.statement) events: timestamped log-like records within a span Context propagation: traceparent header: 00-TRACE_ID-SPAN_ID-FLAGS W3C TraceContext standard: supported by all OTel SDKs Service A passes traceparent to Service B -> same trace_id -> connected spans
```

Tempo Deployment and Configuration

```
# Install Tempo with S3 backend: helm repo add grafana https://grafana.github.io/helm-charts helm install tempo grafana/tempo-distributed \ --namespace observability \ --set storage.trace.backend=s3 \ --set storage.trace.s3.bucket=company-traces \ --set storage.trace.s3.region=us-east-1 \ --set ingester.replicas=3 \ --set querier.replicas=2 \ --set compactor.replicas=1 # OTel Collector exporter to Tempo: exporters: otlp: endpoint: tempo-distributor:4317 tls: insecure: true # Grafana data source for Tempo with trace-to-log correlation: datasources: - name: Tempo type: tempo url: http://tempo-query-frontend:3100 jsonData: tracesToLogsV2: datasourceUid: loki filterByTraceID: true filterBySpanID: false serviceMap: datasourceUid: prometheus
```

CHAPTER 8

Fluent Bit: Log Pipeline Engineering

Fluent Bit is the lightweight, high-performance log processor deployed as a DaemonSet on every Kubernetes node. It collects container logs, enriches them with Kubernetes metadata (pod name, namespace, labels), filters and transforms them, and routes to multiple destinations.

Fluent Bit Pipeline Architecture

```
Fluent Bit pipeline (per node DaemonSet): INPUT FILTER OUTPUT -> Loki tail (container logs) ->
Elasticsearch /var/log/pods/ -> kubernetes -> S3 (long-term archive) (enrich with ->
OpenSearch systemd (node logs) K8s metadata) -> Splunk -> Datadog
```

Fluent Bit ConfigMap for Kubernetes

```
# fluent-bit.conf: [SERVICE] Flush 1 Log_Level info Parsers_File parsers.conf HTTP_Server On
HTTP_Listen 0.0.0.0 HTTP_Port 2020 [INPUT] Name tail Path /var/log/containers/*.log
multiline.parser docker, cri Tag kube.* Mem_Buf_Limit 50MB Skip_Long_Lines On [FILTER] Name
kubernetes Match kube.* Merge_Log On # Parse JSON logs into structured fields Keep_Log Off #
Remove original 'log' field after merge K8S-Logging.Parser On K8S-Logging.Exclude On
Annotations Off # Exclude pod annotations (reduce size) Labels On # Include pod labels
[FILTER] Name grep Match kube.* Exclude kubernetes_namespace_name kube-system # Skip system
logs [OUTPUT] Name loki Match kube.* Host loki-gateway Port 80 Labels
job=fluent-bit,namespace=$kubernetes['namespace_name'],app=$kubernetes['labels']['app']
Batch_Wait 1s Batch_Size 1048576 Line_Format json Remove_Keys kubernetes,stream
```

CHAPTER 9

Elastic Stack on Kubernetes

The Elastic Stack (Elasticsearch, Kibana, Logstash/Fluent Bit, Beats) provides full-text search and analytics over logs, metrics, and security events. ECK (Elastic Cloud on Kubernetes) is the official Operator for deploying Elastic Stack on Kubernetes.

Loki vs Elasticsearch Decision Matrix

Dimension	Loki	Elasticsearch
Indexing	Label-only (low cost)	Full text (high cost, powerful search)
Storage cost	Very low (S3 + compression)	Medium-high (index overhead)
Query language	LogQL (label + regex)	KQL / Lucene (full text, facets)
Cardinality	High label cardinality OK	Index explosion with high cardinality
Search capability	Regex + pattern matching	Full text, fuzzy, aggregations
Kubernetes integration	Native (tail filter)	Via Beats or Fluent Bit
Retention	Object store (cheap, long)	Hot/warm/cold tiers
SIEM capability	Limited	Native (Elastic Security, ECS schema)
Best for	Kubernetes log aggregation	SIEM, compliance, full-text search

CHAPTER 10

SLOs, SLAs, and Error Budgets

Service Level Objectives (SLOs) are the internal targets that define what good service looks like. They are the operational commitment that reliability work is measured against. SLOs translate business requirements into measurable technical targets, and error budgets give teams permission to take risk.

SLO Definitions

```
SLI (Service Level Indicator): The metric being measured. Example: ratio of successful HTTP requests to total requests SLO (Service Level Objective): The target value for the SLI over a time window. Example: 99.9% of requests succeed over a 30-day rolling window SLA (Service Level Agreement): The contractual commitment to customers. Usually more lenient than internal SLO (SLO = 99.9%, SLA = 99.5%) Error Budget: Time allowed to be below SLO. 99.9% SLO = 0.1% error budget = 43.8 min/month acceptable downtime 99.99% SLO = 0.01% = 4.38 min/month 99.999% SLO = 0.001% = 26.3 sec/month
```

Sloth: SLO-as-Code

```
# Sloth generates Prometheus recording rules from SLO definitions: version: prometheus/v1 service: api-server labels: team: platform slos: - name: requests-availability objective: 99.9 description: API server successful request rate sli: events: error_query: | sum(rate(http_requests_total{job='api-server',status=~'5..'}[{{.window}}])) total_query: | sum(rate(http_requests_total{job='api-server'}[{{.window}}])) alerting: name: APIHighErrorRate page_alert: labels: { severity: critical } ticket_alert: labels: { severity: warning } # Generated recording rules: # slo:sli_error:ratio_rate5m (current error rate) # slo:error_budget:ratio (remaining error budget) # Grafana SLO dashboard: burn rate, budget remaining, alerts
```

CHAPTER 11

Alerting Strategy and Runbooks

Effective alerting is harder than collecting metrics. The two failure modes are alert fatigue (too many alerts, on-call ignores them) and alert blindness (not enough alerts, incidents discovered by customers). Good alerting requires discipline: every alert must be actionable, have a runbook, and page someone who can act on it.

Alert Hierarchy

Level	Severity	Response Time	Channel	Example
P1	Critical	Immediate (5 min)	PagerDuty page	API server down; database unreachable
P2	High	30 minutes	PagerDuty notify + Slack	Error rate > 5%; P99 latency > 2s
P3	Medium	Business hours	Slack + Ticket	Error rate > 1%; PVC 80% full
P4	Low	Next sprint	Ticket only	Slow memory growth; minor config drift
Info	Informational	Never pages	Dashboard only	Deployment completed; autoscaling event

Alertmanager Production Configuration

```
# alertmanager.yaml: global: resolve_timeout: 5m slack_api_url:
https://hooks.slack.com/services/XXX inhibit_rules: # If cluster is down, suppress pod-level
alerts: - source_match: alertname: KubernetesClusterDown target_match_re: alertname:
(Pod.*|Node.*|Deployment.*) equal: [cluster] route: receiver: default-slack group_by:
[alertname, cluster, namespace] group_wait: 30s group_interval: 5m repeat_interval: 4h routes:
- match: { severity: critical } receiver: pagerduty-critical repeat_interval: 1h - match: {
team: ai-platform } receiver: ai-platform-slack receivers: - name: pagerduty-critical
pagerduty_configs: - routing_key: XXXXXXXXXXXXXXXX description: '{{ .CommonAnnotations.summary
}}' links: - href: '{{ .CommonAnnotations.runbook_url }}' text: Runbook
```

CHAPTER 12

Cost Observability with OpenCost

Kubernetes cost visibility is a critical operational capability. Without it, teams over-provision resources (wasting money) or under-provision (causing performance issues). OpenCost (CNCF sandbox) provides real-time, workload-level cost allocation that integrates with Prometheus and Grafana.

OpenCost Architecture

```
OpenCost components: OpenCost Deployment: Queries Kubernetes API for resource requests and usage Queries cloud provider billing API for node pricing Computes cost allocation per: pod, namespace, label, team, cluster Exposes metrics via /metrics (Prometheus) and REST API Cost model: Node cost / node resources = cost per CPU-hour, cost per GB-RAM-hour Pod cost = (pod CPU request * CPU cost) + (pod RAM request * RAM cost) Idle cost = (node resources - sum of pod requests) * node cost GPU cost model: GPU node cost / GPUs per node = cost per GPU-hour GPU pod cost = GPU request count * cost per GPU-hour # Install OpenCost: helm repo add opencost https://opencost.github.io/opencost-helm-chart helm install opencost opencost/opencost \
--namespace opencost --create-namespace \ --set opencost.exporter.cloudProviderApiKey=AWS_KEY \ --set opencost.prometheus.external.url=http://prometheus:9090
```

Cost Allocation Queries

```
# OpenCost REST API -- cost by namespace (last 7 days): curl 'http://opencost:9090/model/allocation?window=7d&aggregate;=namespace' # PromQL for cost visibility: # CPU cost per namespace (per hour): sum(kube_pod_container_resource_requests{resource='cpu'}) by (namespace) * on(node) group_left() node_cpu_hourly_cost # GPU cost per pod: sum(kube_pod_container_resource_requests{resource='nvidia_com_gpu'}) by (namespace, pod) * on(node) group_left() node_gpu_hourly_cost # Monthly cost projection per team: sum(opencost:container:cpu_allocation:rate1h) by (label_team) * 720
```

CHAPTER 13

GPU Observability with DCGM

GPU workloads require specialised observability. CPU metrics tell you nothing about whether a GPU is being efficiently utilised for AI training or inference. NVIDIA Data Center GPU Manager (DCGM) provides comprehensive GPU telemetry that integrates with Prometheus and Grafana.

DCGM Exporter Metrics Reference

Metric	Unit	What It Means	Alert Threshold
DCGM_FI_DEV_GPU_UTIL	Percent	SM utilisation; proxy for compute utilisation	Below 50% for training = under-utilisation
DCGM_FI_DEV_MEM_COPY_UTIL	Percent	Memory bandwidth utilisation	Near 100% = memory bandwidth bottleneck
DCGM_FI_DEV_FB_USED	Bytes	GPU memory (VRAM) in use	Above 90% of FB_FREE+FB_USED = OOM risk
DCGM_FI_DEV_POWER_USAGE	Watts	Current power draw	Near TDP = thermal throttle risk
DCGM_FI_DEV_GPU_TEMP	Celsius	GPU die temperature	Above 85C = throttling; above 95C = critical
DCGM_FI_DEV_SM_CLOCK	MHz	SM clock frequency	Drop without thermal = throttle from other cause
DCGM_FI_DEV_XID_ERRORS	Count	GPU error events (Xid codes)	Any Xid = investigate; Xid 79 = GPU crash
DCGM_FI_PROF_GR_ENGINE_ACTIVE	Ratio	Graphics/compute engine active fraction	Low during 'training' = I/O bound
DCGM_FI_PROF_TENSOR_ACTIVE	Ratio	Tensor core utilisation	Low during DL training = poor CUDA kernel

DCGM Exporter Deployment

```
# Install DCGM Exporter as DaemonSet on GPU nodes: helm repo add gpu-helm-charts
https://nvidia.github.io/dcgm-exporter/helm-charts helm install dcgm-exporter
gpu-helm-charts/dcgm-exporter \ --namespace gpu-operator --create-namespace \ --set
serviceMonitor.enabled=true \ --set
serviceMonitor.additionalLabels.release=kube-prometheus-stack # Key DCGM field groups for AI
workloads: # DCGM_FI_PROF_*: Professional metrics (requires DCGM 2.0+, Ampere+) # Including:
GR_ENGINE_ACTIVE, TENSOR_ACTIVE, DRAM_ACTIVE # GPU utilisation alert: alert: GPUUnderutilised
expr: avg_over_time(DCGM_FI_DEV_GPU_UTIL[30m]) < 10 and on(pod)
kube_pod_container_resource_requests{resource='nvidia_com_gpu'} > 0 for: 30m annotations:
```

```
summary: GPU {{ $labels.gpu }} in {{ $labels.pod }} below 10% utilisation for 30min action:
Investigate if training job is stuck or misconfigured
```

CHAPTER 14

AI and LLM Observability

AI and LLM inference workloads require observability beyond standard application metrics. The quality and behaviour of AI responses -- not just the latency of the HTTP response -- must be monitored. This requires a new layer of AI-specific observability integrated into the Kubernetes monitoring stack.

LLM Inference Metrics

Metric	Source	What It Measures	SLO Target
Time to First Token (TTFT)	vLLM/KServe	Latency before streaming starts; user perceived	p50 < 500ms, p99 < 2s
Tokens per second (TPS)	vLLM/KServe	Generation throughput per request	Model-dependent; p50 > 50 tok/s
Inter-Token Latency (ITL)	vLLM/KServe	Time between generated tokens; streaming jitter	p50 < 50ms
Queue wait time	vLLM scheduler	Time request waited before processing	p95 < 1s; scale if exceeded
KV cache hit rate	vLLM	Prefix cache efficiency (reduces compute)	Target > 70% for prod workloads
GPU KV cache utilisation	vLLM	VRAM used for KV cache	Alert if > 95% (request throttling)
Request reject rate	vLLM	Requests dropped due to overload	Any rejection = scale trigger
Token budget exceeded	Application	Requests hitting max_tokens limit	High rate = prompt tuning needed

vLLM Metrics Configuration

```
# vLLM exposes Prometheus metrics natively: # Start vLLM with metrics enabled: python -m vllm.entrypoints.openai.api_server \ --model meta-llama/Meta-Llama-3-70B-Instruct \ --host 0.0.0.0 \ --port 8000 \ --metrics-url /metrics \ --tensor-parallel-size 4 \ --enable-prefix-caching # Key vLLM Prometheus metrics: vllm:e2e_request_latency_seconds (histogram: end-to-end) vllm:time_to_first_token_seconds (histogram: TTFT) vllm:time_per_output_token_seconds (histogram: ITL) vllm:num_requests_running (gauge: active requests) vllm:num_requests_waiting (gauge: queued requests) vllm:gpu_cache_usage_perc (gauge: KV cache utilisation) vllm:gpu_prefix_cache_hit_rate (gauge: prefix cache efficiency) vllm:num_preemptions_total (counter: preempted requests)
```

LLM Observability with OpenTelemetry

```
# OpenLLMetry: OpenTelemetry for LLM applications: # Auto-instruments: OpenAI, Anthropic,
LangChain, LlamaIndex from opentelemetry.sdk.trace import TracerProvider from openllmetry
import Telemetry Telemetry().init() # Auto-instruments LLM calls # Captured span attributes
for LLM calls: # gen_ai.system: openai / anthropic / bedrock # gen_ai.request.model: gpt-4o /
claude-3-opus # gen_ai.request.max_tokens: 4096 # gen_ai.usage.prompt_tokens: 150 #
gen_ai.usage.completion_tokens: 320 # gen_ai.response.finish_reason: stop / length /
tool_calls # These traces appear in Tempo: # Trace: user_request -> agent_run -> tool_call ->
LLM_call # Each LLM call span shows: model, tokens, latency, cost # Agent loop spans show:
planning, tool execution, synthesis
```

CHAPTER 15

Capacity Planning with Observability Data

Observability data is the foundation of capacity planning. Rather than provisioning on intuition or over-provisioning for safety, capacity planning uses historical metrics to predict future needs with confidence intervals.

Capacity Planning Signals

* **CPU headroom**: Identify nodes consistently above 70% CPU utilisation. predict_linear(cpu_usage[7d], 30*24*3600) to project 30-day trend. Scale cluster before reaching saturation.
* **Memory pressure**: Nodes with frequent page cache eviction or swap usage. Containers with memory usage approaching their limits (risk of OOM). Correlate container memory growth with data volume growth.
* **Storage growth rate**: PVCs with predict_linear(usage, 30d) exceeding capacity. Identify which namespaces/workloads are growing fastest. Alert when projected fill date is within 14 days.
* **GPU utilisation trends**: Model serving GPU utilisation by time of day and day of week. Identify peak hours for scaling policies. Project GPU needs for model upgrades (70B model requires 4x the VRAM of 7B model).
* **Network bandwidth**: Identify inter-service communication patterns for network capacity planning. Detect N+1 query patterns (many small requests) vs expected batch patterns.

Capacity Planning Prometheus Queries

```
# Project PVC fill date (which PVCs will fill in < 7 days): predict_linear(
kubelet_volume_stats_used_bytes[6h], 7*24*3600 ) > kubelet_volume_stats_capacity_bytes # Node
CPU saturation trend (which nodes need scaling in 30 days): predict_linear(
instance:node_cpu_utilisation:rate5m[7d], 30*24*3600 ) > 0.85 # GPU memory growth for a model
serving deployment: predict_linear( DCGM_FI_DEV_FB_USED{pod=~'llm-inference.*'}[24h],
7*24*3600 ) / DCGM_FI_DEV_FB_TOTAL > 0.95 # Cluster node count projection based on pod growth:
predict_linear(kube_node_info[7d], 30*24*3600)
```

CHAPTER 16

Observability Anti-Patterns

Anti-Pattern: Alert fatigue -- alerting on symptoms not causes

Problem: Hundreds of alerts fire simultaneously during an incident. On-call engineer cannot identify root cause amid noise. Alerts page for things that auto-recover.

Solution: Alert on SLO burn rate (user impact), not individual metrics. Use inhibition rules. Every alert must have a runbook and be actionable.

Anti-Pattern: High cardinality metric explosions

Problem: Using user IDs, request IDs, or trace IDs as metric labels. 1M users = 1M time series = Prometheus OOM or extreme cost.

Solution: Labels must be low cardinality (max 10,000 unique values per label). Use trace IDs in tracing backends (Tempo), not Prometheus labels.

Anti-Pattern: No trace context propagation

Problem: Services do not pass W3C traceparent headers. Traces are broken: each service shows isolated spans with no connection.

Solution: Enforce trace context propagation via service mesh (Istio/Linkerd) or OTel SDK. Validate with OTel Collector trace sampling.

Anti-Pattern: Logging everything at DEBUG level in production

Problem: Production services logging at DEBUG level generate 100x more logs. Loki ingestion costs explode. Log search becomes slow.

Solution: Production default: INFO or WARN. DEBUG only via dynamic log level adjustment (Zap, logrus, structured logging frameworks). Filter debug logs in Fluent Bit before shipping.

Anti-Pattern: No SLOs -- alerting without user impact context

Problem: Teams track hundreds of internal metrics but cannot answer: Is the service meeting its availability commitment to users?

Solution: Define SLOs for every customer-facing service. Multi-window multi-burn-rate alerts are more actionable than threshold alerts.

CHAPTER 17

Hands-On Exercises

Exercise 10.1 -- Deploy the Full Observability Stack

Install the kube-prometheus-stack and configure Loki:

```
# Add Helm repos: helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts helm repo add grafana
https://grafana.github.io/helm-charts helm repo update # Install kube-prometheus-stack
(Prometheus + Grafana + Alertmanager): helm install kube-prom
prometheus-community/kube-prometheus-stack \ --namespace monitoring --create-namespace \ --set
grafana.adminPassword=admin123 \ --set prometheus.prometheusSpec.retention=7d # Install Loki
stack: helm install loki grafana/loki-stack \ --namespace monitoring \ --set
grafana.enabled=false \ --set promtail.enabled=true # Access Grafana: kubectl port-forward -n
monitoring svc/kube-prom-grafana 3000:80 # Login: admin / admin123 # Explore dashboards:
Kubernetes / Compute Resources / Namespace # Verify metrics: kubectl get servicemonitor -n
monitoring kubectl port-forward -n monitoring svc/kube-prom-kube-prometheus-prometheus 9090 #
Query: count(kube_pod_info)
```

Exercise 10.2 -- Write SLO Alerts

Create a PrometheusRule for availability SLO:

```
# Deploy a sample HTTP application: kubectl create deployment slo-demo --image=nginx:alpine
--replicas=2 kubectl expose deployment slo-demo --port=80 # Create PrometheusRule for 99.9%
availability SLO: kubectl apply -f - <<'YAML' apiVersion: monitoring.coreos.com/v1 kind:
PrometheusRule metadata: name: slo-demo-rules namespace: default labels: release: kube-prom
spec: groups: - name: slo-demo rules: - alert: HighBurnRate expr: | (1 -
rate(nginx_http_requests_total{status!~'5..'}[5m]) / rate(nginx_http_requests_total[5m])) >
0.001 for: 2m labels: severity: critical annotations: summary: SLO burn rate high YAML
```

End of Part X -- Continue to Part XI: Kubernetes for AI Infrastructure

Part XI covers production AI infrastructure on Kubernetes: Kubeflow pipelines, KServe model serving, vLLM distributed inference, Ray and Ray Serve, the NVIDIA GPU Operator, MIG partitioning, distributed training with PyTorch DDP and FSDP, AI gateways and model routing, feature stores with Feast, and production deployment patterns for LLM serving at enterprise scale.